

Automated Game-Day AI Production

Architecture & Implementation Project Map

1. Project Idea & Core Philosophy

To engineer an automated, AI-driven stadium production system that seamlessly manages cameras, music, scoreboard graphics, and commentary. By combining real-time computer vision with authoritative human-in-the-loop triggers, the system aims to replace a sprawling production crew with a highly efficient, single-operator control center.

Design Philosophy: Maximize automation while maintaining strict quality control through a unified dashboard. Edge-optimized AI handles recognition, while specific high-stakes events (like scoring and substitutions) rely on authoritative inputs (the referee and the manager) to guarantee 100% accuracy.

2. System Features

- **CV Jersey Recognition & Camera Tracking:** Automatically identifies players entering the field to trigger personalized walk-up music and dynamic scoreboard graphics.
- **Lineup Validation & Anomaly Rejection:** Cross-references detected jersey numbers with the active roster to filter out false positives (e.g., water runners, coaching staff).
- **Referee Input Terminal:** A low-latency, handheld interface for the umpire/referee to log balls, strikes, runs, and outs, providing absolute ground-truth data for the system.
- **Event-Driven AI Commentary:** Generates real-time, context-aware audio commentary driven by the discrete game-state updates sent from the referee's device.
- **Centralized Manager Dashboard:** A single pane of glass displaying live feeds, system confidence scores, and manual override controls.
- **Confidence-Threshold Alerts:** If CV recognition confidence drops below 70%, the dashboard instantly flags the manager for a manual 3-to-5-second verification.

3. Proposed Technology Stack

To balance the need for ultra-low latency on the field with a robust, scalable control plane, a hybrid stack utilizing optimized microservices is recommended.

Domain	Technologies & Rationale
Computer Vision & Edge Inference	C++ / OpenCV / YOLOv9 or v10: For high-speed frame processing and bounding box tracking. Models can be optimized via TensorRT or ONNX Runtime to run efficiently on local edge devices at the stadium, minimizing network latency.
Backend & Real-Time APIs	FastAPI (Python) & Go: Python (FastAPI) handles the AI orchestration and commentary generation. Go is highly recommended for the concurrent event-streaming layer (WebSockets), processing the high-frequency telemetry from the CV nodes and the referee app with minimal overhead.
Frontend & Dashboard	React & TypeScript: Provides a highly responsive, type-safe single-page application for the manager's dashboard. TypeScript ensures robust data handling for the complex, fast-moving game state.
Infrastructure & Deployment	Docker & Kubernetes / AWS: Containerizing the CV nodes, Go event broker, and Python AI services ensures reliable deployment. Kubernetes can orchestrate these microservices across local stadium servers (edge computing) or scale them via AWS.

4. Implementation Phases

1 Data Collection & Computer Vision Pipeline

- Curate a dataset of baseball players in various lighting conditions and angles.
- Train and fine-tune a YOLOv10 model specifically for jersey number extraction.
- Develop the C++ inference engine to process camera RTSP streams, outputting bounding boxes and class probabilities.

2 State Management & Referee Hardware Integration

- Build the lightweight mobile/hardware app for the referee with a simple button interface (Ball, Strike, Run 1, Run 2, Home Run).
- Develop the high-performance Go-based WebSocket server to ingest this data and instantly update the global game state.
- Implement the roster cross-checking logic to validate players against the database.

3 AI Orchestration & Audio Systems

- Develop the FastAPI microservice that listens to game-state changes and triggers specific music tracks or scoreboard graphics.

- Integrate an LLM/TTS pipeline for the commentary. Prompt the AI with the structured referee data to generate contextual play-by-play audio.

4 Manager Dashboard Development

- Build the React/TypeScript interface.
- Integrate live video feed rendering alongside the digital scoreboard.
- Implement the $< 70\%$ confidence warning system, giving the UI interactive prompts to accept or override the CV prediction within the 5-second window.

5 Containerization & Field Testing

- Wrap all services in Docker containers.
- Write Kubernetes manifests to manage network routing between the CV edge nodes, the central database, and the user interfaces.
- Conduct a simulated game test to calibrate the camera pan/tilt/zoom responses and test the latency of the manager's override capabilities.